Method	Queries	Key selection	β	Values	Nonuniform
H2O+	context-prefill	HighestAttnKeys	none	direct	none
SnapKV (agnostic)	mock-question	HighestAttnKeys	none	direct	none
PyramidKV (agnostic)	mock-question	HighestAttnKeys	none	direct	linearly decreasing
KVzip-uniform	repeat-prefill	HighestAttnKeys	none	direct	none
KVzip	repeat-prefill	GlobalHighestAttnKeys	none	direct	global top-k

Table 4. **Baseline configurations.** "Mock-question" denotes a single synthetic question generated via self-study to instantiate originally question-aware methods in a question-agnostic setting. "Direct" values indicate that retained keys keep their original values (i.e., no value fitting with least squares).

methods below): instead of duplicating KV heads before compaction (as some implementations do), we aggregate queries from all query heads that attend to a given KV head into a shared query set for that KV head.

SnapKV and PyramidKV are question-aware in their original form (they compact after observing the downstream question). To make them question-agnostic and comparable to our setting, we instead generate a single mock question via self-study and use it to construct the query set used for key scoring. We use max-pooling with a kernel-size of 7, consistent with original hyperparameters.

Table 4 summarizes the design choices for each baseline.

DuoAttention. For Llama-3.1-8B-Instruct, we additionally attempted to compare against DuoAttention (Xiao et al., 2025), which supports this model. We implement DuoAttention by partitioning heads into *streaming* and *retrieval* heads: for streaming heads we evict the article portion of the KV cache, while for retrieval heads we retain the full KV cache. The overall compaction ratio determines the fraction of heads assigned to each category. However, at the compaction ratios we evaluated (retaining up to $0.2\times$ the original cache), DuoAttention performed poorly, so we omit it from the main figure.

Summarization. As a baseline, we summarize the article text and use the summary in place of the original article for downstream evaluation. We evaluate several prompt variants shown in Table 5.

Name	Prompt

`summarize`	`Summarize the following text:`
	`{article_text}`
	`Summary:`

`summarize_indepth`	`Summarize the following text in depth:`
	`{article_text}`
	`Summary:`

`summarize_keypoints_questions`	`Summarize the following text, providing all key points that might be necessary to answer comprehension questions:`
	`{article_text}`
	`Summary:`

`summarize_concise`	`Summarize the following text concisely:`
	`{article_text}`
	`Summary:`

`summarize_very_concise`	`Summarize the following text very concisely:`
	`{article_text}`
	`Summary:`

Table 5. **Summarization prompts.**

C. Algorithmic Implementation Details

We implement each of our per-head algorithm sequentially, iterating through each KV-head in the model. We did not find major speedups through batching computation across heads.

C.1. OMP Speedups

The periodic-refit OMP key selection algorithm is shown in Algorithm 2. It introduces two hyperparameters, k and τ: k is the number of keys selected per greedy iteration, and τ is the number of iterations between NNLS refits (i.e., we recompute weights once every $k\tau$ newly added keys). Our main algorithm implicitly uses $k = 1, \tau = 1$, and we use $k = 4$ and $\tau = 2$ in our fast variant.

Algorithm 2 OMP Keys Selection (Periodic Refit)

Require: Original keys $\boldsymbol{K} \in \mathbb{R}^{T \times d}$, queries $\boldsymbol{Q} \in \mathbb{R}^{n \times d}$, budget t, top-k k, NNLS interval τ
Ensure: Indices S, weights $\mathbf{w}$ (where $\boldsymbol{\beta} = \log \mathbf{w}$)

$\boldsymbol{\Phi}_{ij} \leftarrow \exp\left(\frac{1}{\sqrt{d}} \boldsymbol{q}_i \boldsymbol{K}_j^\top\right)$ {Mass feature matrix}
$\mathbf{m}_i \leftarrow \sum_{j=1}^{T} \boldsymbol{\Phi}_{ij}$ {Target mass vector}
$S \leftarrow \emptyset, \quad \mathbf{w} \leftarrow [\,], \quad \mathbf{r} \leftarrow \mathbf{m}$
for $u = 1$ **to** t **do**
  $c_j \leftarrow \mathbf{r}^\top \boldsymbol{\Phi}_{:j} \quad \forall j \notin S$ {Correlation scores}
  $J^\star \leftarrow \mathrm{TopK}\big(\{c_j\}_{j \notin S},\, k\big)$ {Select top-k new keys}
  $S \leftarrow S \cup J^\star$
  if $u \bmod \tau = 0$ **or** $u = t$ **then**
    $\mathbf{w} \leftarrow \arg\min_{\mathbf{w} \geq 0} \|\boldsymbol{\Phi}_{:S}\mathbf{w} - \mathbf{m}\|_2^2$ {Solve NNLS}
    $\mathbf{r} \leftarrow \mathbf{m} - \boldsymbol{\Phi}_{:S}\mathbf{w}$ {Update residual}
  end if
end for
return $S, \mathbf{w}$

C.2. Linear Algebra Subroutines

Numerics. We compute $\boldsymbol{C}_k$, $\boldsymbol{\beta}$, and $\boldsymbol{C}_v$ in FP32, then cast to BF16 for storage and subsequent use.

Least Squares. We evaluated three solvers for the least-squares problems in Eq. 3: `torch.linalg.lstsq`, `torch.linalg.pinv`, and a Cholesky-based approach using `torch.linalg.cholesky` with `torch.cholesky_solve`. Concretely, we compute

1. `torch.linalg.lstsq`: We directly compute the solution to the optimization problem of $\arg\min_M \|XM - Y\|_F^2$, as $M = \texttt{torch.linalg.lstsq}(X, Y)$. On CUDA, PyTorch uses the `gels` driver, performing QR decomposition on $X = QR$, transforming the target Y by left multiplication with $C = Q^T$, then solves the triangular problem of $RM = C$.

2. `torch.linalg.pinv`: We compute the Moore-Penrose pseudo-inverse explicitly and then right multiply by Y, i.e. $M = \texttt{torch.linalg.pinv}(X)\,Y$.

3. `torch.linalg.cholesky`: We solve the normal equations $X^T X M = X^T Y$ by computing the Cholesky decomposition of the symmetric positive-definite matrix $A = X^T X$:

$$LL^T = X^T X$$

   where $L = \texttt{torch.linalg.cholesky}(A)$. The solution is obtained by solving the two-step triangular system $LZ = X^T Y$ and $L^T M = Z$ via `torch.cholesky_solve`.

Across our experiments, solution quality ranked `lstsq` > `cholesky` > `pinv`, though the difference was minimal. In terms of runtime, `cholesky` was fastest, followed by `lstsq`, then `pinv`. We therefore use `torch.linalg.lstsq`

in all experiments. We also tested ℓ_2 regularization when estimating $\boldsymbol{\beta}$ and $\boldsymbol{C}_v$ by computing the regularized solution $(X^T X + \lambda I)^{-1}$, but found that it degraded performance across all positive values of λ.

Nonnegative Least Squares (NNLS). We implement an NNLS solver with projected gradient descent. When `iters=0`, we compute an unconstrained least-squares solution with `torch.linalg.lstsq` and then clamp to enforce $B \geq \epsilon$ (and optionally $B \leq u$). For `iters>0`, we warm-start from this clamped solution and run projected gradient descent for `iters` steps on $\frac{1}{2}\|MB - y\|_2^2$, using a fixed step size $1/L$ where $L \approx \|M\|_2^2$ is estimated via a few power-iteration steps.

Algorithm 3 describes the NNLS solver used throughout. We initialize from an unconstrained least-squares solution followed by clamping, and (when `iters>0`) refine the solution using projected gradient descent on $\frac{1}{2}\|MB - y\|_2^2$ with a fixed step size.

Algorithm 3 NNLS via Projected Gradient Descent

Require: Matrix $M \in \mathbb{R}^{n \times p}$, target $y \in \mathbb{R}^n$, iterations `iters`, lower bound $\epsilon \geq 0$, optional upper bound u
Ensure: Nonnegative weights $B \in \mathbb{R}^p$

$B^{(0)} \leftarrow \arg\min_B \|MB - y\|_2^2$ {Unconstrained least squares}
$B^{(0)} \leftarrow \max(B^{(0)}, \epsilon)$ {Clamp to enforce $B \geq \epsilon$}
if upper bound u is specified **then**
  $B^{(0)} \leftarrow \min(B^{(0)}, u)$
end if
if $\texttt{iters} = 0$ **then**
  return $B^{(0)}$
end if
Estimate $L \approx \|M\|_2^2$ via power iteration
$\eta \leftarrow 1/L$ {Fixed step size}
for $t = 0$ **to** `iters` **do**
  $g^{(t)} \leftarrow M^\top(MB^{(t)} - y)$ {Gradient}
  $\tilde{B} \leftarrow B^{(t)} - \eta g^{(t)}$ {Gradient step}
  $B^{(t+1)} \leftarrow \max(\tilde{B}, \epsilon)$ {Projection}
  if upper bound u is specified **then**
    $B^{(t+1)} \leftarrow \min(B^{(t+1)}, u)$
  end if
end for
return $B^{(\texttt{iters})}$

For OMP Keys, we set $\texttt{iters} = 0$ as we observed that the unconstrained solution rarely yields negative weights. For Highest Attention Keys, we use `iters=2`.

Stabilizing $\boldsymbol{\beta}$. Our pipeline first fits $\boldsymbol{\beta}$ to match attention mass and then fits $\boldsymbol{C}_v$ to match attention outputs. A potential failure mode of this two-stage procedure is that mass matching can assign extremely small weights to some selected keys (i.e., very negative $\boldsymbol{\beta}$, or effectively $\boldsymbol{\beta} = -\infty$). Such keys may have little effect on the mass objective yet still be useful for reducing attention-output error; however, once $\boldsymbol{\beta}$ is very negative, the corresponding key cannot contribute to the attention output, regardless of $\boldsymbol{C}_v$.

To mitigate this, we apply simple stability constraints on $\boldsymbol{\beta}$. For *Highest Attention Keys*, we replace NNLS with a bounded least-squares over $w = \exp(\boldsymbol{\beta})$, enforcing $e^{-3} \leq w_j \leq e^3$ (equivalently, $\boldsymbol{\beta}_j \in [-3, 3]$). The projected gradient method above supports these box constraints directly.

For *OMP Keys*, which rarely produces extreme $\boldsymbol{\beta}$ values, we instead use a lightweight pruning rule: after greedy selection, we discard any selected keys with $\boldsymbol{\beta} < -7$ and continue selection until all retained keys satisfy $\boldsymbol{\beta} \geq -7$. We also cap large biases during weight fitting by enforcing $w_j \leq e^7$ (equivalently, $\boldsymbol{\beta}_j \leq 7$). In practice, these OMP safeguards did not yield noticeable performance gains, since OMP rarely assigns very low or high $\boldsymbol{\beta}$ values; we nonetheless include them as simple, theoretically sound stabilizers.

C.3. Chunked Compaction Details

Here we provide implementation details for chunked compaction. Given a document split into N contiguous chunks, we apply per-layer/per-head compaction independently to each chunk and concatenate the resulting compacted KV segments to form the final cache:

$$[\text{prefix}] \;+\; [\text{compacted chunk}_1] \;+\; \cdots \;+\; [\text{compacted chunk}_N] \;+\; [\text{suffix}],$$

where the prefix and suffix are kept fixed with $\boldsymbol{\beta} = 0$, and each chunk contributes learned $(\boldsymbol{C}_k, \boldsymbol{\beta}, \boldsymbol{C}_v)$.

We consider two variants: KV-based and text-based chunking.

KV-based chunking prefills the full context once, slices out each chunk's KV states, compacts those tensors, and concatenates the compacted chunks. To generate self-study reference queries for chunk i, we construct a cache by concatenating the corresponding KV tensors:

$$[\text{prefix}] \;+\; [\text{chunk}_i] \;+\; [\text{suffix}],$$

leaving any sliding-window layers unchanged (copied from the full prefill). When prefilling self-study conversations to extract reference queries, we continue positional indexing from the original sequence length so that appended tokens receive the same RoPE phases as in the uncompacted run. These choices preserve the global positional semantics of the original sequence: although the physical cache size is smaller, its logical length and RoPE phases match those of the uncompacted run.

Text-based chunking prefills and compacts each chunk in isolation as $[\text{local prefix}] + [\text{local chunk}_i] + [\text{local suffix}]$, using chunk-local positions starting at 0, and then applies a uniform RoPE phase shift to the compacted keys to align them to the chunk's original global offset in the full context (i.e., a rotation by $\Delta = p_{\text{global}} - p_{\text{local}}$). For models with sliding-window attention (e.g., Gemma), we handle sliding layers by instantiating the final cache's sliding-window KV from the last chunk (including the global suffix) with the same positional shift applied.

C.4. Self-study

In our implementation of self-study, we run two "instances" of the model:

1. **Conversation-starter generation.** Instance A is prompted with $\mathcal{C}$ and a seed prompt (e.g., "Write 3 different questions separated by newlines that test understanding of the context."). We sample

   $$a \sim \text{LM}(\cdot \mid \mathcal{C}, \text{seed}),$$

   then post-process a into m separate conversation starters $(a_i)_{i=1}^m$.

2. **Response generation.** For each a_i, we sample from instance B a response

   $$b_i \sim \text{LM}(\cdot \mid \mathcal{C}, a_i).$$

   During instance B's prefill on a_i and decoding of b_i, we record, for each attention head, all query vectors used by the model.

We include a set of fixed prompts (e.g., "Aggregate all key facts mentioned in the context.") by directly setting a to the prompt string and sampling the corresponding response from instance B.

We use vLLM (Kwon et al., 2023) to generate self-study conversations, and then we prefill these conversations with a forward hook that extracts query vectors at each KV-head. To control the number of extracted queries, we apply reservoir sampling and cap the buffer at $50{,}000$ query vectors per KV-head. In practice, our self-study configuration typically yields fewer than this cap (avg. $\sim 16{,}000$ per head for QuALITY).

For on-policy self-study and repeat-prefill, we first generate each self-study sequence and prefill the first layer to extract its queries. Then, for each layer L, we prefill layer L using the compacted cache for layers 0 through $L-1$, and extract the query vectors at layer L.